

```

fun main() {
    fun one(n: Int): List<Int> {
        return if (n <= 0) {
            emptyList()
        } else {
            (n downTo 1).toList()
        }
    }

    fun two(n: Int): List<String> {
        val pyramid = mutableListOf<String>()
        for (i in 1..n) {
            val spaces = " ".repeat(n - i)
            val hashes = "#".repeat(2 * i - 1)
            val level = spaces + hashes + spaces
            pyramid.add(level)
            println(level) // Вывод в консоль
        }
        return pyramid
    }

    val pyramidLevels = 3
    val pyramid = two(pyramidLevels)
    println("\nPyramid as a list:")
    pyramid.forEach { println(it) }

    fun tre(text: String, key: Int): String {
        val shift = key % 26 // Нормализация ключа (если key > 26)
        return text.map { char ->
            when {
                char.isUpperCase() -> {
                    val shifted = (char.code - 'A'.code + shift) % 26
                    'A' + shifted
                }
                char.isLowerCase() -> {
                    val shifted = (char.code - 'a'.code + shift) % 26
                    'a' + shifted
                }
                else -> char // Не буквы остаются без изменений
            }
        }.joinToString("")
    }

    println("Введите текст для шифрования:")
    val text = readln()
    println("Введите ключ (сдвиг):")
    val key = readln().toInt()

    val encrypted = tre(text, key)
    println("Зашифрованный текст: $encrypted")

    val decrypted = tre(encrypted, 26 - key)
    println("Расшифрованный текст: $decrypted")

    fun fizzBuzz(n: Int): List<String> {
        return (1..n).map { num ->
            when {
                num % 15 == 0 -> "БизлБизл"
                num % 3 == 0 -> "Физлл"
                num % 5 == 0 -> "Бизллл"
                else -> num.toString()
            }
        }
    }

```

```
        }.toList()
    }

    println("Введите число n:")
    val n = readln().toInt()
    val result = fizzBuzz(n)
    println(result)
}
```